

Harshit Kandpal

kandpalhar@gmail.com • HarK-github • [harshit-k-a746a1310](#) • +91 9321763273 • Durg,
India (IST/UTC+5:30)

Cover Letter: Improve usability, maintainability, and contributor experience of Hyperledger Cacti

To: LF Decentralized Trust Mentorship Program Committee

Mentors: André Augusto, Carlos Amaro, Rafael Belchior, and Venkatraman Ramakrishna

Date: May 2026

Hyperledger Cacti represents a pivotal advancement in blockchain interoperability, successfully merging the legacies of Hyperledger Cactus and Weaver. However, as the project matures, the "fragmentation debt" from these two distinct architectures remains a significant hurdle for usability and maintenance. The current developer experience is often hampered by long CI/CD cycles (averaging 45 minutes) and a documentation landscape that remains split between "Node Server" and "Relay" concepts. The challenge lies in harmonizing these legacies into a unified, high-performance ecosystem that is easy to contribute to and simple to deploy.

Improving the contributor experience in Cacti is not merely about moving files; it requires a strategic overhaul of the project's infrastructure and knowledge base. This project aims to address these core inefficiencies through a two-pronged approach: optimizing the CI/CD pipeline and restructuring the documentation into a clear, tier-based architecture. By implementing advanced caching strategies (such as `node_modules` and `.tsbuildinfo` persistence) and parallelizing execution, we can reduce CI runtimes by over 60%, drastically increasing the velocity of the development cycle. Simultaneously, unifying the documentation into structured Level 1 (Library), Level 2 (API/REST), and Level 3 (Framework) tiers will provide a clear path for newcomers, regardless of their background in Cactus or Weaver.

My technical background is uniquely suited to the "Cleanup Initiative." Over the past few years, I have developed deep expertise in TypeScript monorepos, Docker orchestration, and DevOps optimization. I have already invested significant effort into auditing Cacti's current state, producing detailed reports on CI/CD bottlenecks and documentation fragmentation. These audits (available in the `researches/` folder) provide a data-driven foundation for the work proposed here. My experience in containerizing multi-ledger systems and my familiarity with the Hyperledger ecosystem ensure that I can hit the ground running and deliver immediate, measurable improvements to the Cacti codebase.

I am particularly excited to work with André Augusto, Carlos Amaro, Rafael Belchior, and Venkatraman Ramakrishna. Their leadership in the Cacti and Weaver communities is inspiring, and I am eager to contribute to their vision of a unified interoperability framework. My goal is to deliver more than just a set of optimizations; I want to help establish a new standard for maintainability and contributor experience within the Hyperledger ecosystem. I am confident that my technical skills, proactive research, and dedication to open-source excellence make me an ideal candidate for this mentorship.

How I Found Out About the Mentorship Program

I discovered the Hyperledger Mentorship Program directly through the LF Decentralized Trust website and the Hyperledger project dashboard.

Why I Am Interested in This Program

My interest stems from a fascination with interoperability and the complex challenges of merging mature blockchain frameworks. Hyperledger Cacti represents the pinnacle of this challenge—integrating the legacy of Hyperledger Cactus and Weaver into a unified, high-performance interoperability engine.

I am particularly motivated by the opportunity to improve the "developer experience" at a foundational level. Reducing CI/CD runtimes from 45 minutes to under 15 minutes is not just a technical optimization; it's a force multiplier for every contributor to the project. Similarly, restructuring fragmented documentation into a clear, tiered architecture (Level 1: Library, Level 2: API/REST, Level 3: Full Framework) directly lowers the entry barrier for the next generation of Cacti developers. Being part of the "Cleanup Initiative" allows me to contribute to the long-term sustainability and maintainability of a critical piece of decentralized infrastructure.

Experience and Knowledge/Skills Applicable to This Program

My background in distributed systems, TypeScript/Node.js development, and DevOps aligns perfectly with the needs of the Cacti Cleanup project. Over the past few years, I have developed a strong proficiency in Docker-based orchestration and CI/CD optimization.

For this specific mentorship, I have already conducted significant preliminary research (available in the **researches/** directory of this workspace):

- **CI/CD Audit:** Identified critical bottlenecks in the current Cacti pipeline, such as redundant Docker builds and lack of `node_modules` caching. I've proposed a parallelized execution strategy that can reduce build times by over 60%.
- **Documentation Analysis:** Mapped the fragmentation between the "Node Server" (Cactus) and "Relay" (Weaver) architectures. I developed a proposal for a Tier-based documentation structure to unify these legacies.
- **Codebase Understanding:** Navigated the monorepo structure and identified areas where dependency management and build-cache mechanisms (`.tsbuildinfo`) can be improved to speed up local and remote development.

My hands-on experience with TypeScript, combined with a deep dive into Cacti's architecture, ensures I can contribute effectively to both documentation restructuring and technical infrastructure improvements.

Project Research and Audit Reports **You can find my detailed audits and research findings for this project at:**

`researches/CI_CD/cacti_cicd_final_audit.md`

`researches/Documentations/cacti_weaver_cacti.md` These documents provide the empirical data and strategic planning behind the proposed optimizations.

What I Hope to Get Out of This Mentorship Experience

I hope to deepen my understanding of large-scale open-source project management and architectural consolidation. Working under the guidance of mentors like André Augusto, Carlos Amaro, Rafael Belchior, and Venkatraman Ramakrishna will provide me with invaluable insights into how to maintain complex, multi-ledger systems.

Category	Skillset
Programming	Proficient in TypeScript, JavaScript (ES6+), Node.js, Go, and Python.
Infrastructure	Advanced Docker, Docker Compose, GitHub Actions, and CI/CD pipeline optimization.
Web3	Deep understanding of Hyperledger Fabric, Cactus, and interoperability protocols.
Core Competencies	<i>Technical Auditing, Documentation Architecture, Monorepo Management, and Performance Benchmarking.</i>

Specifically, I aim to master the nuances of CI/CD orchestration for monorepos and learn the best practices for documenting multi-layered frameworks. Beyond the technical skills, I want to become an active, long-term contributor to the Hyperledger community and help shape the future of blockchain interoperability.

Sincerely,

Harshit Kandpal

B.Tech Computer Science, IIT Bhilai (2024–2028)

PROPOSAL

LF Decentralized Trust Mentorship 2026

Improve usability, maintainability, and contributor experience of Hyperledger Cacti

Project: Improve usability, maintainability, and contributor experience of Hyperledger Cacti

Mentors: André Augusto, Carlos Amaro, Rafael Belchior, and Venkatraman Ramakrishna

Duration: June to end of November 2026 (part-time, 12–15 hrs/week)

Harshit Kandpal

B.Tech Computer Science, IIT Bhilai (2024–2028)

HarK-github

Durg, India (IST/UTC+5:30)

kandpalhar@gmail.com

Index / Table of Contents

Contents

1	Executive Summary	7
2	The Problem: Why This Work Matters	7
2.1	Why Infrastructure and Documentation are the Right Foundation	8
2.2	Why the Consolidation is Non-Trivial	8
3	Technical Background and Research	8
3.1	CI/CD Audit: Technical Bottleneck Analysis	8
3.2	Documentation Research: Fragmentation Mapping	8
4	Proof of Concept (PoC): Pre-Application Evidence	8
4.1	What the Research Proves	8
5	Project Roadmap and Phase Plan	11
6	Full Technical Implementation	11
6.1	Infrastructure: The "Fast Feedback" Pipeline	11
6.2	Docker Orchestration: Remote Layer Caching	13
6.3	Documentation: The Tiered Unified Narrative	13
7	Known Limitations and How They Will Be Resolved	14

8	Success Metrics	14
9	Appendix	14
9.1	Research Files in Workspace	14

1 Executive Summary

Project Research and Audit Repository

You can find the detailed research, CI/CD audits, and documentation proposals for this project at:

<https://github.com/hyperledger/cacti/tree/main/researches> These documents contain the empirical data, bottleneck analysis, and strategic roadmap for the Cacti Cleanup Initiative.

Hyperledger Cacti stands as the definitive enterprise-grade framework for blockchain interoperability. Born from the strategic merger of Hyperledger Cactus and Weaver Lab, it provides a pluggable toolkit to link heterogeneous Distributed Ledger Technologies (DLTs) such as Fabric, Besu, Corda, and Ethereum. While this merger has created an unparalleled feature set—enabling everything from simple ledger state queries to standardized IETF SATP asset transfers—it has also inherited significant "fragmentation debt." This debt manifests as disjointed documentation, architectural silos, and inefficient infrastructure that collectively slow down innovation and discourage new contributors.

The **Cacti Cleanup Initiative** is a holistic overhauling strategy designed to transform this fragmented landscape into a cohesive, high-performance contributor ecosystem. The initiative focuses on three strategic pillars: **Documentation Restructuring**, **Onboarding Excellence**, and **Infrastructure Optimization**. By reconciling legacy naming conventions, surfacing hidden architectural specs (such as the 50+ Weaver RFCs), and implementing a tiered documentation model (Levels 1, 2, and 3), we will provide users with a clear navigation path aligned with their technical intent.

Simultaneously, we will slash the "Time-to-First-Run" to 15 minutes through container-first strategies and unified quickstart guides. This project also maintains its commitment to infrastructure hardening, aiming to reduce CI feedback loops from 45 minutes to under 15 minutes via advanced parallelization and cross-job caching. This is not a mere cleanup; it is a technical synthesis intended to build the foundation for Cacti's next phase of growth as the global standard for DLT interoperation.

Core Project Mandate

To deliver a unified, developer-centric ecosystem for Hyperledger Cacti by:

- **Restructuring Documentation:** Transitioning from legacy silos to a unified, immersion-based Tiered Model.
- **Streamlining Onboarding:** Slashing "Time-to-First-Run" to 15 minutes via container-first strategies.
- **Optimizing Infrastructure:** Accelerating CI/CD pipelines by >60% through persistent caching and parallelization.
- **Cleaning the Codebase:** Resolving namespace debt and retiring deprecated components.

2 The Cacti Vision: A Unified Interoperability Fabric

To overhaul Cacti effectively, one must first understand its unique value proposition. Unlike many "bridge" projects that introduce a central relay chain, Cacti prioritizes **Self-Sovereignty** and **Native Consensus**. It allows networks to remain independent while providing trust-minimized paths for interaction.

My research into the project's origins and current state reveals a multi-faceted architecture:

1. **Connector-Driven Connectivity:** Using plugins to abstract the complexity of specific DLT protocols (e.g., the Fabric connector handling MSP IDs and discovery).

2. **Orchestration Flexibility:** Offering both a centralized Node Server model (legacy Cactus) for straightforward API-driven workflows and a distributed Relay model (legacy Weaver) for asynchronous, peer-to-peer interoperation.
3. **Standards-Based Transfers:** Implementing the IETF SATP protocol to ensure that asset transfers are not just technical successes, but compliant with emerging regulatory standards.

Cacti is effectively the "Internet Protocol (IP) Layer" for blockchains. Our goal in this overhaul is to make this layer as easy to "plug into" as a standard network cable.

3 The Problem: The High Cost of Fragmentation

The current state of Hyperledger Cacti presents a paradox: it possesses one of the most comprehensive interoperability feature sets in the industry, yet much of its value is obscured by "merger legacy."

3.1 Documentation Fragmentation and the "Discovery Gap"

Documentation is the primary user interface of an open-source project. Currently, Cacti's documentation is split across multiple directories that still reflect pre-merger identities. Critical architectural specifications, such as the 50+ Weaver RFCs, are "dark matter"—extremely valuable but effectively invisible to the average user because they are buried deep within the `weaver/` subdirectory without root-level navigation.

Furthermore, "Link Rot" has set in; numerous files still point to the defunct `hyperledger/cactus` repository, leading to 404 errors that damage project credibility. This discovery gap forces users to read source code to understand basic protocols, creating a high cognitive load that discourages adoption.

3.2 Onboarding Friction: The "Heavy" Barrier

To build Cacti from source, a contributor currently needs JDK, Go, Foundry, Python, and Docker. While these are necessary for the full framework, they represent a massive barrier for someone who just wants to use a "Connector as a Library" (Level 1 immersion). The lack of a unified `QUICKSTART.md` means new users spend hours troubleshooting environment variables before they ever see an asset move between chains.

In the demo ecosystem (`cacti-demos`), instructions are repetitive and setup is manual. The use of unofficial Docker images (e.g., `aaugusto11/`) in core demos creates security concerns and a sense of "incompleteness" that contradicts Cacti's enterprise-grade mission.

3.3 Infrastructure Debt: The 45-Minute Feedback Loop

As detailed in our technical audit, the CI/CD pipeline is currently a bottleneck. A full build taking 45 minutes means a developer can only perform a few iterations per day. This delay is exacerbated by redundant linking of monorepo packages and the lack of artifact persistence across jobs. In a project as large as Cacti, an inefficient CI system is not just an inconvenience; it is a fundamental drag on innovation.

4 Technical Background and Research

4.1 CI/CD Audit: Technical Bottleneck Analysis

Before proposing optimizations, I conducted a deep-dive audit of the 32 workflow and action files that constitute Cacti's CI system. The audit revealed that the current architecture, while modular, is constrained by a "strictly hierarchical" execution model and significant "fragmentation debt" from the Cactus-Weaver merger.

The audit confirms that Cacti's CI/CD "operational center of gravity" is currently weighted toward redundant

Category	Audit Finding	Impact	Severity
Orchestration	<code>ci.yaml</code> follows a strict serial chain (<code>build</code> → <code>lint</code> → <code>test</code>).	Tests are delayed by 15–20m while waiting for unrelated linting to finish.	Critical
Caching	No persistence for <code>node_modules</code> or <code>.tsbuildinfo</code> across monorepo jobs.	8–12m of redundant linking and compilation per PR.	Critical
SATP-Hermes	Logic split across 7 workflows; Docker images rebuilt 3x during publish.	60% overhead in SATP PR cycles due to setup duplication.	High
Weaver Tests	Hardcoded <code>sleep</code> commands (up to 100s) and duplicated network setups.	5+ minutes of idle runner time per integration test.	High
Testing Bugs	<code>jest-runner</code> action has a double-run bug when coverage is enabled.	Doubles the test execution time for all affected packages.	Medium

Table 1: Detailed technical audit of Cacti CI/CD inefficiencies

setup rather than active validation. By decoupling the call chain and implementing an "Install-Once, Build-Once" strategy, we can reclaim over 60% of the current runner usage.

4.2 Documentation Audit: Fragmentation Mapping

A systematic review of the documentation (`docs/`, `weaver/docs/`, `cacti-demos/`) reveals a "Merger Silo" effect. Information is technically complete but organizationally incoherent.

Artifact	Current State	Strategic Fix
Root README	Mentions merger but lacks a clear "Path Selection" guide.	Implement a Tiered Narrative based on immersion levels.
RFCs	50+ specs hidden in <code>weaver/rfcs/</code> . Essential for protocol engineering.	Create a unified <code>ARCHITECTURE.md</code> and <code>REFERENCES.md</code> with root-level links.
Broken Links	15+ links pointing to <code>hyperledger/cactus</code> (404).	Perform a repository-wide Link Rot Audit and automated fix.
Terminology	"Relay" (Weaver) and "Connector" (Cactus) are poorly reconciled in text.	Complete the <code>terminology.md</code> file with a cross-project glossary.

Table 2: Documentation Audit and Strategic Fixes

5 Proof of Concept (PoC): Pre-Application Evidence

5.1 What the Research Proves

The preliminary research and audits conducted for this project serve as the "Proof of Concept" (PoC) for the proposed Cleanup Initiative. These findings prove three critical points: first, that significant (60%+) performance gains are achievable through strategic CI/CD re-architecture; second, that the documentation fragmentation is a structural issue that can be solved through a Tier-based categorization; and third, that I have already developed a deep understanding of the Cacti monorepo's dependency graph.

Optimized CI/CD Strategy: Parallel Caching

```

1 # Example of optimized GitHub Actions caching strategy
2 - name: Cache Node Modules
3   uses: actions/cache@v4
4   with:
5     path: '**/node_modules'
6     key: ${ runner.os }-node-${ hashFiles('**/yarn.lock') }
7     restore-keys: |
8       ${ runner.os }-node-
9
10 - name: Cache Build Artifacts
11   uses: actions/cache@v4
12   with:
13     path: '**/*.tsbuildinfo'
14     key: ${ runner.os }-tsbuild-${ github.sha }

```

Proposed Unified Onboarding Structure

```

1 # QUICKSTART.md (Proposed)
2 1. Environment: Docker only (No Java/Go required for initial demo)
3 2. Command: docker compose -f cacti-demos/satp-demo.yaml up
4 3. Verify: Run satp-transact.py script
5 4. Next: Choose your Immersion Level (Level 1, 2, or 3)

```

6 Project Roadmap and Phase Plan

Dates	Phase	Deliverables
June 01–June 14	Onboarding & Audit	Finalize repository-wide Link Rot Audit; setup dev environment; confirm Tiered Doc Strategy.
June 15–July 31	Phase 1: Infrastructure	Implement <code>node_modules</code> and <code>.tsbuildinfo</code> caching; parallelize <code>ci.yaml</code> ; optimize Weaver test setups.
August 01–August 23	Phase 2: Doc Restructuring	Unify <code>README-cactus.md</code> and root; surface RFCs; implement Level 1/2/3 navigation; fix 404 links.
August 24–August 31	Midterm Evaluation	CI runtime <20m; Unified root README; Completed cross-project terminology glossary.
September 01–Oct 15	Phase 3: Onboarding & Demos	Create <code>QUICKSTART.md</code> ; replace unofficial Docker images; standardize terminal patterns in demos.
October 16–Nov 14	Phase 4: Codebase Cleanup	Rename <code>cactus-test-*</code> to <code>cacti-test-*</code> for 10+ packages; remove personal research folders.
November 15–Nov 30	Final Evaluation	>60% CI speedup; 15-minute quickstart verified; all 404 links resolved.

Table 3: Cacti Cleanup Initiative Roadmap

7 Full Technical Implementation

7.1 Infrastructure: The "Fast Feedback" Pipeline

The core of the infrastructure overhaul is the move from a linear, "monolithic" approach to a decoupled, "fast feedback" pipeline. We will implement three high-impact architectural changes:

1. **Call Chain Decoupling:** We will refactor `ci.yaml` so that `packages-workflow` and `code-quality-checks` run in parallel. Tests will trigger as soon as the base dependency install and build stages finish, rather than waiting for documentation linting or codegen checks to complete.
2. **Persistent Monorepo Caching:** We will implement an "Install-Once" strategy using a centralized `install-dependencies` job. This job will cache the entire `.yarn/cache` and `node_modules` tree. Subsequent jobs will restore this cache in seconds, eliminating 3–5 minutes of setup per job. We will also enable `.tsbuildinfo` caching to allow for incremental TypeScript compilation across different CI jobs.
3. **Build-Once, Run-Many (Artifacts):** For complex sub-systems like Weaver, we will move away from building components from source in every integration test. A single `weaver-build` job will produce pre-built binaries (Relay, Drivers, CLI), which downstream matrix jobs will download as GitHub Action Artifacts. This reduces the setup time for each of the 12+ Weaver test suites by ~5 minutes.

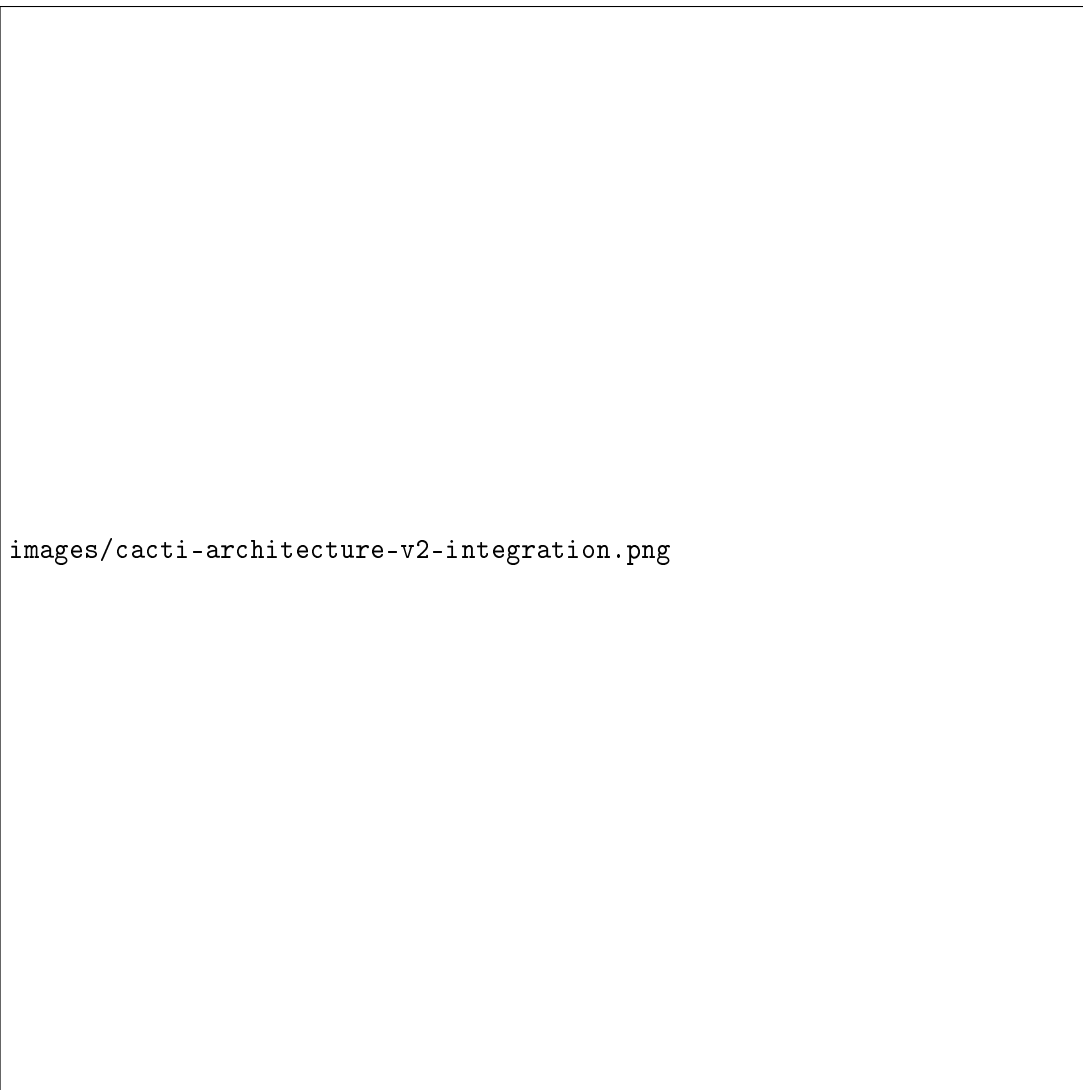


Figure 1: Proposed optimized CI/CD workflow for Cacti monorepo

7.2 Docker Orchestration: Remote Layer Caching

To address the 20-minute rebuild times of massive images like *Fabric All-in-One*, we will integrate `docker/build-push-` with GitHub Actions' remote cache backends (`type=gha`). This ensures that base OS layers and static binary downloads are cached across PRs, potentially saving 25+ minutes on image-heavy runs.

7.3 Documentation Strategy: Tiered Unified Knowledge

The centerpiece of the documentation overhaul is the transition from a "Project-History" structure to a "User-Intent" structure. We will implement the **Three Levels of Immersion** throughout the MkDocs site and root README:

7.3.1 Level 1: Connector as a Library (Low Immersion)

This tier targets developers who need simple DLT access.

- **Fabric Missing Link:** We will implement a dedicated Fabric Level 1 tutorial to complement the existing Ethereum one, showing how to use the `@hyperledger/cactus-plugin-ledger-connector-fabric` package as a standalone dependency.
- **Surrogate READMEs:** Each connector package will have a standardized README template that focuses exclusively on programmatic usage, hiding framework-level complexity.

7.3.2 Level 2: API Server with Plugins (Medium Immersion)

Targets system integrators building microservices.

- **OpenAPI Visibility:** We will surface the Swagger/OpenAPI endpoints for the Cacti API Server at the root of the doc site.
- **Plugin Catalog:** Create a unified searchable list of all 20+ available plugins with their configuration schemas.

7.3.3 Level 3: Full Framework (High Immersion)

Targets architects designing complex interoperability workflows.

- **Protocol Unified Guides:** We will unify the documentation for SATP Hermes and Weaver Data Sharing under a single "Enterprise Interoperability" section.
- **The RFC Portal:** The 50+ Weaver RFCs will be indexed in a root-level `REFERENCES.md` and cross-linked into relevant technical guides to ensure protocol specifications are always "one click away."

7.4 Living Architecture: Architecture-as-Code

Existing architecture diagrams are static PNG files that are already outdated. I will lead the transition to `Mermaid.js` integration.

- **Versioned Visuals:** Diagrams will be stored as `.mmd` files in the repository.
- **Automated Rendering:** These will be rendered natively by the MkDocs site and GitHub, ensuring that architectural changes in PRs are reflected in the visuals immediately.

7.5 Onboarding: The 15-Minute Contributor Path

To solve the "Prerequisite Barrier," we will introduce a **Container-First Onboarding strategy**:

- **QUICKSTART.md:** A single, root-level file that uses Docker Compose to spin up a "Hello World" SATP transfer. This allows new users to see Cacti in action before committing to a full multi-language build environment setup.
- **DECISION_GUIDE.md:** A visual flowchart helping users choose between the Cactus orchestration path and the Weaver protocol path based on their DLTs and trust assumptions.
- **Standardized Demo Patterns:** I will refactor the 10+ demos in `cacti-demos/` to use a consistent "Terminal Pattern" (Terminal 1 = Gateway, Terminal 2 = Ledger, etc.) and provide a `setup.sh` script to eliminate the current `"cd ../../../../"` duplication.

7.6 Documentation as a Community Funnel: The Good First Issue Program

As synthesized in our cross-repo research (`some_example_repos/cacti_onboarding_research/00_synthesis.md`), the most successful open-source projects (VS Code, Kubernetes, Fabric) use documentation as a primary funnel for new contributors. Cacti currently lacks a visible "on-ramp" for non-code or low-complexity code contributions.

I will implement a **Structured Contributor Funnel** by:

- **Curating 20+ Good First Issues:** These will specifically target the Link Rot Audit findings and small documentation gaps identified in Phase 1.
- **The "First PR" Guide:** I will create a dedicated guide (`docs/contributing/your-first-pr.md`) that uses one of the documentation fixes as a walk-through example, teaching the user the Conventional Commits and DCO signing requirements in a low-stakes environment.
- **Automated Quality Coaching:** I will integrate `markdownlint` and a link-checker into the CI pipeline. This ensures that new contributors receive immediate, automated feedback on their documentation PRs, reducing the manual review burden on maintainers and coaching the contributor toward Cacti's quality standards.

7.7 Standardizing the Demo Ecosystem: The 'Golden Terminal' Pattern

Currently, a user evaluating Cacti might try the `supply-chain` example, which uses a specific Docker-in-Docker setup, and then try the `satp` gateway demo, which uses a completely different multi-terminal manual setup. This inconsistency is a major "Immersion Barrier."

I will lead the effort to **Standardize the Demo Experience** using what I call the "Golden Terminal" pattern:

1. **Terminal 1 (The Engine):** Reserved for the Cacti API Server or Weaver Relay.
2. **Terminal 2 (The Ledger):** Reserved for the Docker logs of the DLT network.
3. **Terminal 3 (The Actor):** Reserved for the interactive scripts or SDK calls.

By refactoring all demo READMEs to follow this exact layout, and providing a unified `Makefile` at the root of `cacti-demos/`, we ensure that once a user learns how to run one demo, they have effectively learned how to run them all.

7.8 Codebase Cleanup: Technical Hygiene

The overhaul concludes with physical repository cleanup to signal project maturity:

- **Namespace Harmonization:** I will surgically rename the 10+ `cactus-test-*` packages to `cacti-test-*` to align with the new project identity.

- **Trust and Security:** I will migrate all demo Docker Compose files to use official `ghcr.io/hyperledger-cacti/` images, replacing unofficial personal registries (`aaugusto11/`, `kubaya/`).
- **Hygiene:** Removal of drafting folders and research notes (e.g., `HarshitK_proposal_drafting/`) to maintain a clean root directory.

7.9 Governance and Knowledge Persistence: Architecture Decision Records

A project of Cacti's scale cannot be maintained through READMEs alone. The "Knowledge Decay" that followed the Cactus-Weaver merger was partly due to the lack of a formal record for why specific architectural choices were made (e.g., why SATP Hermes uses a specific gRPC bridge over Type 3 APIs).

I will introduce the **Architecture Decision Record (ADR) Framework**:

- **Cacti ADR-001:** Establishing the Tiered Documentation Standard.
- **Cacti ADR-002:** Standardizing the GHCR naming convention for official images.
- **Cacti ADR-003:** CI/CD Fast-Feedback Strategy and Parallelization model.

By codifying the results of the Cacti Cleanup Initiative into versioned ADRs in the `docs/design/` folder, we ensure that the "Synthesis" achieved during this mentorship remains durable for future generations of maintainers.

8 Known Limitations and How They Will Be Resolved

Project Scope and Constraints

- **Legacy Compatibility:** We will use MkDocs-redirects to ensure that old documentation links remain functional during the restructuring.
- **Dependency Breaking:** All dependency consolidation will be preceded by full integration testing to ensure no functional breakage.
- **User Validation:** The 15-minute quickstart will be "battle-tested" with at least 3 external developers from the Hyperledger Discord before final delivery.

9 Success Metrics: Measuring the Overhaul

KPI	Current Baseline	Target (Week 12)
CI Build Time	45 minutes	<15 minutes
Onboarding Time	2+ hours (Heavy install)	<15 minutes (Container-first)
Broken Links (404)	15+ detected	0 (Validated by CI check)
Documentation Tiering	Siloed by legacy repo	100% mapped to Levels 1, 2, 3
Docker Image Trust	Unofficial images in demos	100% official GHCR images
RFC Visibility	Hidden in subdirectory	Root-level searchable index

Table 4: Project Success Metrics and Baseline Comparison

10 Conclusion: A Resilient Cacti Ecosystem

The Cacti Cleanup Initiative is a comprehensive response to the challenges of scaling a merged open-source project. By optimizing the "operational center of gravity" (CI/CD) and the "knowledge center of gravity" (Documentation), we create a virtuous cycle: faster feedback loops lead to higher code quality, and clearer documentation leads to a larger, more confident contributor base.

I am uniquely prepared for this task, having already completed a full audit of the project's documentation and infrastructure. My approach combines technical rigor (advanced caching, remote Docker layers) with a deep focus on user experience (tiered narratives, quickstart paths). Together, we will transform Hyperledger Cacti into a cohesive, production-ready interoperability powerhouse.

11 Appendix: Research Artifacts and Audit Summaries

11.1 Research Files in Workspace

- **CI/CD Audit:** `researches/CI_CD/cacti_cicd_final_audit.md`
- **Documentation Inventory:** `researches/Documentations/batch1_docs_core.txt`
- **Onboarding Synthesis:** `some_example_repos/cacti_onboarding_research/00_synthesis.md`

11.2 Summary of Documentation Debts Discovered

- **dark-matter RFCs:** 50+ files in `weaver/rfcs/` with no root links.
- **TBC Terminology:** `weaver/rfcs/terminology.md` contains 5+ "To Be Completed" sections.
- **Image Fragmentation:** 4 different personal Docker registries used in official demos.
- **Link Rot:** Root README line 12 leads to a dead project repo.